58. Cron Jobs: Run Code on a Schedule (Linux, NestJS and Databases)

Builds on **#49 (Redis)**, **#53 (Kubernetes)**, **#56 (Node async)** and **#57 (BullMQ)**.

> **In one line:** a **cron job** is a task that runs **automatically at a fixed time or interval**, like "every night at 2 AM, delete expired OTPs", without anyone clicking a button.

Real-life example: the alarm clock

> You set an alarm: **"6:00 AM, Monday to Friday"**. Every weekday the alarm rings by itself. You don't press anything.
> **Alarm clock** = the **cron scheduler** (it watches the time).
> **"6:00 AM, Mon-Fri"** = the **cron expression** (the schedule).
> **"Wake up and go to the gym"** = the **job** (the code that runs).

Where cron jobs are used every day

Job	Schedule
Delete expired OTPs / sessions from the DB	Every 10 minutes
Send a "daily sales report" email to the admin	Every day at 9:00 AM
Take a database backup	Every night at 2:00 AM
Generate monthly invoices / salary slips	1st of every month at 00:00
Remind students "fees due tomorrow"	Every day at 6:00 PM
Refresh a cached leaderboard / report table	Every 5 minutes

A little history

- **cron** comes from the Greek word **"chronos" = time**.
- It was born in **Unix (1970s, Bell Labs)**. The modern version most Linux systems use is **Vixie cron (1987)** by Paul Vixie.
- The schedule format ("5 stars") became so popular that today **Node, Java, Kubernetes, AWS, PostgreSQL** all use the same syntax.

PART 1: How cron works

1. The cron daemon (the always-awake watchman)

On Linux, a background program called `crond` (the **cron daemon**) is always running. It reads a list of jobs called the **crontab** (cron table).

```
every minute, crond wakes up
        |
        v
reads the crontab:  "0 2 * * *  backup.sh"
        |
        v
does the current time match the pattern?
    |                       |
   NO                     YES
    |                       |
sleep till next minute   start the job (new process)
                            |
                         output -> log / email
```

Key idea: cron does not "count down". Every minute it simply asks: **"does NOW match any pattern?"** If yes, it runs that job.

2. The cron expression (the 5 stars)

```
┌──────────── minute        (0 - 59)
│ ┌────────── hour          (0 - 23)
│ │ ┌──────── day of month  (1 - 31)
│ │ │ ┌────── month         (1 - 12)
│ │ │ │ ┌──── day of week   (0 - 6, 0 = Sunday)
│ │ │ │ │
* * * * *   command-to-run
```

Symbol	Meaning	Example
`*`	every value	`* * * * *` = every minute
`,`	list	`0 9,18 * * *` = 9 AM and 6 PM
`-`	range	`0 9 * * 1-5` = 9 AM, Mon to Fri
`/`	step ("every N")	`*/10 * * * *` = every 10 minutes

Cheat sheet

Expression	Runs
`* * * * *`	Every minute
`*/5 * * * *`	Every 5 minutes
`0 * * * *`	Every hour, at minute 0
`0 2 * * *`	Every day at 2:00 AM
`30 9 * * 1-5`	9:30 AM, Monday to Friday
`0 0 1 * *`	Midnight on the 1st of every month
`0 0 * * 0`	Every Sunday at midnight

> **6-field version:** Node libraries (`@nestjs/schedule`, `node-cron`) also allow an extra **seconds** field at the front: `*/30 * * * * *` = every 30 seconds. Linux cron and most databases only have 5 fields (minimum = 1 minute).

> Not sure about an expression? Paste it into **crontab.guru**. It tells you in plain English when it runs.

3. Linux crontab example

```
# open your crontab for editing
crontab -e

# m  h  dom mon dow  command
0    2  *   *   *    /home/app/backup.sh >> /var/log/backup.log 2>&1
*/10 *  *   *   *    curl -s http://localhost:3000/jobs/cleanup

# list your jobs
crontab -l
```

`>> file 2>&1` saves the output and errors to a log file, so you can see later whether the job worked.

PART 2: Cron jobs in NestJS

4. Setup: @nestjs/schedule

```
npm install @nestjs/schedule
```

```
// app.module.ts
import { ScheduleModule } from '@nestjs/schedule';

@Module({
  imports: [
    ScheduleModule.forRoot(),   // turns on the scheduler
    TypeOrmModule.forRoot({ /* postgres config */ }),
    TasksModule,
  ],
})
export class AppModule {}
```

5. Real example: clean expired OTPs + daily report

We have an `otp` table in Postgres. Each OTP is valid for 5 minutes. Old OTPs pile up, so we delete them every 10 minutes. We also email a student count every morning.

```
// tasks/tasks.service.ts
import { Injectable, Logger } from '@nestjs/common';
import { Cron, CronExpression, Interval, Timeout } from '@nestjs/schedule';
import { InjectRepository } from '@nestjs/typeorm';
import { LessThan, Repository } from 'typeorm';
import { Otp } from '../otp/otp.entity';
import { Student } from '../student-pg/student.entity';

@Injectable()
export class TasksService {
  private readonly logger = new Logger(TasksService.name);

  constructor(
    @InjectRepository(Otp) private otpRepo: Repository<Otp>,
    @InjectRepository(Student) private studentRepo: Repository<Student>,
  ) {}

  // 1) every 10 minutes: delete OTPs that already expired
  @Cron('*/10 * * * *')
  async cleanExpiredOtps() {
    const result = await this.otpRepo.delete({ expiresAt: LessThan(new Date()) });
    this.logger.log(`Deleted ${result.affected} expired OTPs`);
  }

  // 2) every day at 9:00 AM India time: daily report
```

```
  @Cron('0 9 * * *', { name: 'dailyReport', timeZone: 'Asia/Kolkata' })
  async dailyReport() {
    const total = await this.studentRepo.count();
    this.logger.log(`Daily report: ${total} students`);
    // await this.mailService.send('admin@school.com', `Students: ${total}`);
  }

  // 3) built-in constants instead of writing stars
  @Cron(CronExpression.EVERY_DAY_AT_MIDNIGHT)
  midnight() { this.logger.log('New day started'); }

  // 4) every 30 seconds (not a cron expression, just milliseconds)
  @Interval(30_000)
  healthPing() { this.logger.debug('still alive'); }

  // 5) once, 5 seconds after the app starts
  @Timeout(5_000)
  warmUp() { this.logger.log('warm up cache once at startup'); }
}
```

Decorator	Meaning	Real-life
`@Cron('0 9 * * *')`	At a **clock time** (pattern)	Alarm at 9 AM
`@Interval(30000)`	**Every N ms** since the app started	Drink water every 30 minutes
`@Timeout(5000)`	**Once**, N ms after start	"Remind me in 5 minutes"

Start, stop or change a job at runtime

```
import { SchedulerRegistry } from '@nestjs/schedule';
import { CronJob } from 'cron';

constructor(private registry: SchedulerRegistry) {}

stopReport() {
  this.registry.getCronJob('dailyReport').stop();     // pause
}

addDynamicJob() {
  const job = new CronJob('0 18 * * *', () => this.logger.log('6 PM fee reminder'));
  this.registry.addCronJob('feeReminder', job);
  job.start();
}
```

Without NestJS (plain Node): node-cron

```
import cron from 'node-cron';

cron.schedule('*/10 * * * *', async () => {
  await db.query('DELETE FROM otp WHERE expires_at < NOW()');
```

```
  console.log('cleaned');
}, { timezone: 'Asia/Kolkata' });
```

PART 3: Cron jobs inside databases

Sometimes the job is **only SQL** (delete old rows, refresh a report table). Then the database itself can run it. No Node app is needed, and the job keeps running even if your API is down.

6. PostgreSQL: pg_cron

Postgres has no scheduler built in. The **pg_cron extension** adds one (it is available on AWS RDS, Azure, Supabase and Google Cloud SQL).

```
-- postgresql.conf:  shared_preload_libraries = 'pg_cron'
CREATE EXTENSION pg_cron;

-- every 10 minutes: delete expired OTPs
SELECT cron.schedule(
  'clean-otps',
  '*/10 * * * *',
  $$ DELETE FROM otp WHERE expires_at < NOW() $$
);

-- every night at 2 AM: refresh a report
SELECT cron.schedule('refresh-report', '0 2 * * *',
  $$ REFRESH MATERIALIZED VIEW daily_student_report $$);

SELECT * FROM cron.job;                    -- list jobs
SELECT * FROM cron.job_run_details         -- history: success / failed
  ORDER BY start_time DESC LIMIT 10;
SELECT cron.unschedule('clean-otps');      -- remove a job
```

7. MySQL / MariaDB: Event Scheduler (built in)

```
SET GLOBAL event_scheduler = ON;

CREATE EVENT clean_otps
  ON SCHEDULE EVERY 10 MINUTE
  DO
    DELETE FROM otp WHERE expires_at < NOW();

CREATE EVENT monthly_invoices
  ON SCHEDULE EVERY 1 MONTH
  STARTS '2026-12-01 00:00:00'
  DO
    CALL generate_invoices();        -- a stored procedure
```

```
SHOW EVENTS;
DROP EVENT clean_otps;
```

MySQL does not use the 5-star syntax. It uses `EVERY 10 MINUTE`, `EVERY 1 DAY`, and so on.

8. MongoDB: no built-in cron (3 options)

Option	How	Best for
TTL index (auto-delete)	Mongo deletes documents by itself after a time	OTPs, sessions, logs. No cron needed at all!
Atlas Scheduled Triggers	In MongoDB Atlas (cloud), a function runs on a cron schedule	Reports and cleanup on Atlas
App-side cron	`@Cron` in NestJS with Mongoose	Self-hosted Mongo

```
// TTL index: delete each OTP 300 seconds after createdAt
db.otps.createIndex({ createdAt: 1 }, { expireAfterSeconds: 300 })

// Mongoose schema version
@Schema()
export class Otp {
  @Prop() code: string;
  @Prop({ default: Date.now, expires: 300 }) createdAt: Date;   // TTL 5 min
}
```

> The Mongo TTL monitor runs about **every 60 seconds**, so a document may live up to about a minute longer than the TTL. That's fine for OTPs.

9. Redis: no cron, but two helpers

- **Key TTL** (see #54): `SET otp:9876 4821 EX 300`. Redis deletes the key after 300 seconds. For expiry, you don't need a cron.
- **BullMQ repeatable jobs** (see #57): a schedule that is **stored in Redis** and run by workers. Only **one** worker runs each tick, even if 10 pods are running.

```
// BullMQ: schedule a repeatable job (stored in Redis)
await this.reportQueue.upsertJobScheduler(
  'daily-report',
  { pattern: '0 9 * * *', tz: 'Asia/Kolkata' },
  { name: 'send-report', data: {} },
);
```

10. Other databases (quick look)

Database	Scheduler	Example
SQL Server	**SQL Server Agent** jobs	A job with a step (T-SQL) and a schedule (daily 2 AM), made in SSMS
Oracle	**DBMS_SCHEDULER**	`repeat_interval => 'FREQ=DAILY;BYHOUR=2'`
SQLite	None	Use app-side cron or Linux cron
Cassandra / DynamoDB	None, but has **TTL** on rows/items	Auto-expire data. Use the app or AWS for the rest.

11. Comparison: which database, how

DB	Built-in scheduler?	Auto-expire (TTL)?	Syntax
PostgreSQL	Via the **pg_cron** extension	No (use cron + DELETE)	5-star cron
MySQL	**Yes**, Event Scheduler	No (use an EVENT)	`EVERY 1 DAY`
MongoDB	Atlas Triggers only	**Yes**, TTL index	5-star (Atlas)
Redis	No (BullMQ adds one)	**Yes**, `EX` / `EXPIRE`	BullMQ pattern
SQL Server	**Yes**, Agent	No	GUI / T-SQL
Oracle	**Yes**, DBMS_SCHEDULER	No	`FREQ=DAILY`

PART 4: Cron in production

12. Where should the cron live?

Place	Good when	Watch out
App (`@Cron`)	The job needs app logic (send email, call an API)	Runs in **every** copy of the app
Database (pg_cron, EVENT)	The job is pure SQL	Business logic hidden in the DB is harder to test and review
Linux crontab	A single server, scripts, backups	Lost if the server is replaced
Kubernetes CronJob	The app runs in K8s (#53)	A new pod starts each time (startup cost)
AWS EventBridge Scheduler	Serverless: triggers Lambda / ECS (#51)	Cloud cost, AWS-specific
BullMQ repeatable	Many pods, and only one should run the job	Needs Redis

Kubernetes CronJob example

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: db-backup
spec:
  schedule: "0 2 * * *"
  timeZone: "Asia/Kolkata"
  concurrencyPolicy: Forbid          # don't start if the last run is still going
  successfulJobsHistoryLimit: 3
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: backup
              image: postgres:16
              command: ["sh", "-c", "pg_dump $DATABASE_URL > /backup/db.sql"]
```

13. The big trap: duplicate runs

> You scaled your NestJS API to **3 pods** (#53). Each pod has `@Cron('0 9 * * *')`.
> At 9:00 AM **all 3 pods** fire, so the admin gets **3 report emails** and invoices are generated **3 times**!



Fixes

1. **Run cron in only one place:** a separate "scheduler" app with 1 replica, or a K8s CronJob.
2. **BullMQ repeatable job:** Redis makes sure only one worker gets each run.
3. **Lock** (Redis or Postgres): the first pod to grab the lock runs, and the others skip.

```
// Redis lock: only one pod wins
@Cron('0 9 * * *')
async dailyReport() {
  const got = await this.redis.set('lock:dailyReport', process.env.HOSTNAME,
                                   'EX', 300, 'NX');   // NX = only if not exists
  if (!got) return;                                     // another pod has it
  await this.sendReport();
}

-- Postgres version: advisory lock
SELECT pg_try_advisory_lock(42);   -- true for one session only
```

14. Other production rules

Problem	Example	Fix
Time zone	Server is UTC: `0 9 * * *` runs at 2:30 PM India time	Set `timeZone: 'Asia/Kolkata'`, or calculate in UTC
Overlap	The job takes 15 minutes but runs every 10, so 2 copies run	A "running" flag/lock, K8s `concurrencyPolicy: Forbid`
Silent failure	Backup failing for a month, nobody knew	Log every run, alert on errors, health checks (e.g. a ping URL)
Not idempotent	A retry charges fees twice	Make jobs safe to run twice (check "already done?")
Too big at once	Deleting 10 million rows locks the table	Delete in batches: `LIMIT 1000` in a loop
Heavy work	Sending 1,000 emails inside `@Cron` blocks the API	Cron only **adds jobs** to BullMQ, and workers do the work (#57)
Server was off	The PC was asleep at 2 AM, so the job was missed	cron doesn't catch up. Use `anacron` or a "last run" check.

15. Summary

- **Cron job** = code that runs automatically on a schedule (an alarm clock for code).
- **Expression** = 5 fields: `min hour day month weekday`. `*/10 * * * *` = every 10 minutes.
- **NestJS:** `ScheduleModule.forRoot()` + `@Cron` / `@Interval` / `@Timeout`.
- **Databases:** Postgres = **pg_cron**, MySQL = **EVENT**, Mongo = **TTL index** / Atlas Triggers, Redis = **EX** / BullMQ, SQL Server = Agent, Oracle = DBMS_SCHEDULER.
- For simple **expiry** (OTP, sessions), prefer **TTL** (Mongo / Redis) over a cron.
- In production, watch out for **duplicate runs** across pods, **time zones**, **overlaps** and **silent failures**.
- Heavy work: **cron schedules, the queue does the work** (cron + BullMQ + KEDA).